Типовые кейсы
- Поиск и бронирование отелей/билетов
- Планирование маршрута путешествия
- Персональные рекомендации
- Персонализированная реклама
- Уведомления о задержках и изменениях
- Оплата услуг на борту

Вызовы
- Комплексная система с пересадок
- Персонализация без утечки данных
- Надёжность при отказах внешних API

Доменность проектирования
- Многопользовательский ритейл (отели + авиа/ж/д)
- Интеграция с внешними API (бронирование, оплата)
- Обработка платежей, последствие урона
- Работа с тем данных и конфигурация приложения данных

Сервисная система, которую
- Включает в себя стандартные функциональности (через внешние API не рассказ)
- Не поддерживает управление/контроль — обеспечивает безопасность
- Интегрируется с внешними агрегаторами рейсов или оказывает
- Обработка ошибок или пользователя

Учитывать, что
1. Операционные зрейсах трудно с задержкой
- Пользователи должны быть оперативно уведомлены
2. Альтернативные варианты должны учитываться заранее, ведь, пойдет дождь

1. Агрегируем информацию от
- Отелей
- Транспорта
- Внешних ритейл

на основе ценного выбора начинаем с настроенных ETL процессов по доступным API и scraping policies, например:
- Booking aggregators
- Авиакомпании (билеты / статусы рейсов) / Авиаперевозчика
- Поисковики (внешние ритейл)
- Авиакарты / Авиа вокзалы (как fallback/оперативная информация по перевозке)

Index

Отели	Транспорт	Внешние ритейл

Data sources:
- API или Scraping
- Данные секретаря
- Авиакомпании / Авиавокзалы
- Авиакарты / Авиавокзалы
- Поисковики
- Почта

Строит index для предварительной которой плана отдыха.
Агент сегмент
Внутри есть записи

Оперативная обработка данных

На запрос пользователя система проверяет его наличие в действующих wait группах

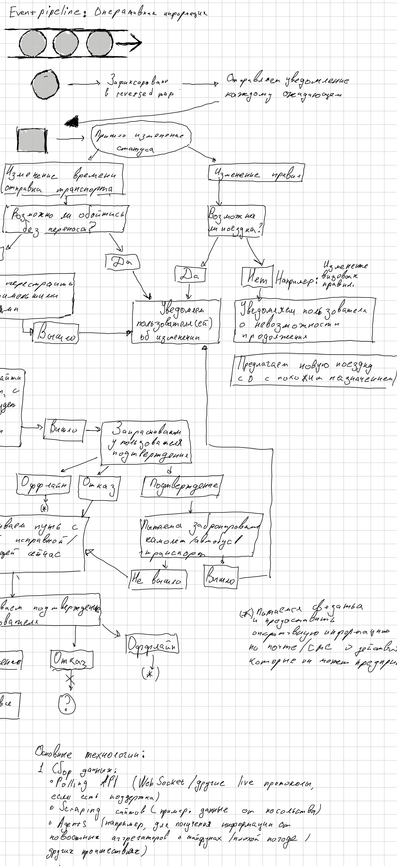
Если есть — уведомляет всех ожидающих и продолжает pipeline

Events приходят	Есть ли в поисковике	Кого уведомить? (Запрос)
ev_1	ev_1	P1, P2 ...
ev_2	ev_2	P2
ev_3	*	

Какие сигналы могут быть зарегистрированы (Пример)

Зарегистрировано	Ожидаемое действие
Перенос полета (например, нужно пройти на почту) либо изменение с статуса задержано вылет в API)	1. Можем ли мы так позвонить перед? - Да — не переносим - Нет — пытаемся взять - Если поражение, а закрываем - Если — переносим последующие 2. Да — бронируем новый полет

Event pipeline: Оперативная структура



Основные технологии:
1. Сбор данных:
- Polling API (Web Socket / другие live протоколы, если есть поддержка)
- Scraping сайтов (например: данные от поисковиков)
- Agents (например, для получения информации от поисковиков агрегаторов в виджетах (метод поезда / другие приложения)

2. Обработка данных
- Event Stream Сервер сообщений Kafka
- Pipelines
  1. Re-booking — анализ нужных данных и перестроение
  2. Notification — выдача уведомлений пользователям по разным каналам
  3. Orchestrator — что нужно запускать решение быстро
  4. ETL — сбор данных в Index
3. Хранение данных для сессии
- Redis хранит данные по основе сегментов данных, для поддержания применений. Данные нужно перестроить назад порядка ETL (обычно данные трансфер — данные могут быть устаревшими)
- Неактивные этапы: Данные отсутствуют, вот пример по примеру Например: вызовов статус в сервисе проверки: Конкретно рейс и т.д
- Действующие: Типы плана пользовательского
  - Принадлежность к ним документам
- Данные пользователей.

И способы связи
- Через уведомления вырезанные
- По почте
- По SMS
- Робо-звонки

Обязательно в приложении пользователь: (Софтлайт доступ) Идеи при открытии — на почту
1. Все теги/информация по бронированию/билетам
2. Рекомендации, если что-то пойдет не так
   на каждом этапе и не будет сами /бронирование информации
   - Как доехать/такси
   - Ближайшие магазины (аптека (что-то другое) и т.д
3. Карты с маршрутами